

Data centers

prepared for EGOI 2022 by Cheng Zhong

Subtask #1

Sorting the available machines in each of the n data centers and output the result directly, as there's no service to be launched.

- Time complexity: $O(n^2)$ or $O(n \log n)$

Subtask #2 & #3

Simulate the process. Using an array or vector to store the number of machines in each data centers. Using some $O(n^2)$ (subtask #2) or $O(n \log n)$ (subtask #3) sorting algorithm to sort the data centers in descending order before launching each service, and deduct m_i machines in each of the top c_i data centers.

- Time complexity: $O(n^2 s)$ or $O(n (\log n) s)$

Subtask #4

Instead of using quick sort or other comparison based sorting algorithms, using counting sort as there are at most 1000 machines in each data centers.

- Time complexity: $O(1000 s)$

Subtask #5

As there's only 1 data center being affected after each service is launched, we could move it to a proper position in $O(n)$ time and keep the array still in descending order.

- Time complexity: $O(n (\log n) + ns)$

Alternatively, instead of using an array or vector to store the number of machines, using a priority queue. As c_i is always 1, while launching each service, we only need to visit and change the root, so we only need $O(\log n)$ time to deal with each service.

- Time complexity: $O((n + s) (\log n))$

Subtask #6

After launching each service, the data centers can be divided into two parts: one just deducted m_i machines each, another remains untouched. Each part remains its descending order. Therefore, we can merge these two parts like that in merge sort, and it only takes $O(n)$ time to deal with each new service.

- Time complexity: $O(n (\log n) + ns)$

Motivation of providing this problem

Sorting algorithms has great importance and is introduced in the very first chapters in *Introduction to Algorithms* and many other books. However, some students only learn that the best time complexity for comparison sort is $O(n \log n)$, and the "sort" in C++ library provides such functionality. They don't really understand what's behind all these sorting algorithms. This task tries to help the students thinking about the stories behind the basic sorting algorithms, and how to take advantage of each of them.